Shipment API: Project Documentation

- Shipment API — Interview Guide
  - 1. Your 30-second pitch
  - 2. What happens when a customer creates a shipment
  - 3. Each concept: where it is and what to say
  - 4. Live demo script (curl)
  - 5. “What would you change for production?”
  - 6. Be honest about it
- Setup & API Reference (README)
  - 1. Requirements
  - 2. Setup
  - 3. Run it
  - 4. Run the tests
  - 5. API reference
  - 6. Using it from Angular
  - 7. Project map
  - 8. Switching to Redis + Horizon (production)

Shipment API — Interview Guide

This guide explains the project in simple words: what each part does, where it is, and what to say when an interviewer asks about it.

---

1. Your 30-second pitch

> “I built a small shipment booking API in Laravel 13. A customer creates a shipment, the system calculates the price, and a background job books it with a courier company. Every status change is saved in a history table and the customer gets an email.
>
> I kept it to one feature, but used the parts a real production system needs: Sanctum token login, policies for permissions, global, group and route middleware, Form Request validation, a custom service provider that binds an interface to a class, a queued job with retries, an observer, and events with sync and queued listeners. The API is designed for an Angular front end.”

---

2. What happens when a customer creates a shipment

This is the most important thing to explain. Follow one request from start to end.

```
POST /api/shipments   (Authorization: Bearer <token>)
 |
 |  1. AssignRequestId          GLOBAL middleware: gives the request an ID for logs
 |  2. ForceJsonResponse        API GROUP middleware: errors always come back as JSON
 |  3. auth:sanctum             ROUTE middleware: checks the token, loads the user (401 if bad)
 v
 4. StoreShipmentRequest        authorize(): asks ShipmentPolicy::create()   -> 403 if false
                                rules(): validation                          -> 422 if invalid
 v
 5. ShipmentController::store   The container injected ShipmentService into the constructor
 v
 6. ShipmentService::create
      - RateCalculator works out the price
      - DB::transaction starts
      - $user->shipments()->create(...)
            |
            +-> ShipmentObserver::creating()   adds tracking number + 'pending' status
            +-> INSERT into shipments
            +-> ShipmentObserver::created()    fires ShipmentStatusChanged(null -> pending)
                    +-> RecordStatusHistory     (sync)   writes a history row now
                    +-> SendStatusNotification  (queued) waits for the commit
      - BookShipmentWithCourier::dispatch()->afterCommit()
      - transaction COMMITS  -> the job and the listener are pushed to the queue
 v
 7. 201 Created + ShipmentResource JSON      (the response goes back through the middleware)

Later, in the queue worker:
 8. BookShipmentWithCourier::handle(CourierGateway $courier, ShipmentService $shipments)
      - is it still pending? (idempotent check)
      - $courier->book()      -> FakeCourierGateway or HttpCourierGateway (chosen by the provider)
      - changeStatus(Booked)  -> save() -> Observer::updated() -> event again
                                 -> new history row + a "Booked" email
```

---

3. Each concept: where it is and what to say

3.1 Authentication — Laravel Sanctum

Files: `AuthController.php`, `routes/api.php`, `User.php` (uses `HasApiTokens`)

How it works: On login, we check the password with `Hash::check()` and create a token with `createToken()`. Sanctum stores a **hash** of the token in `personal_access_tokens`. Angular sends the token in the `Authorization: Bearer` header on every request. The `auth:sanctum` middleware finds the token and loads the user. Logout deletes only the current token, so other devices stay logged in.

Say: "I used Sanctum API tokens because the client is an Angular SPA and maybe a mobile app later. Passport would only be needed for full OAuth2 with outside companies."

Follow-up questions:

- *Why does login return a 422 and not a 401 for a wrong password?* — It's a form error the user can fix, so it's shown under the email field. 401 means "no valid token".
- *How do you stop brute force?* — `throttle:5,1` on the login route: 5 tries per minute.
- *Why is `role` not fillable?* — So nobody can register as an admin by sending `"role": "admin"`. That's mass assignment protection.

3.2 Middleware — global, group and route

Files: `app/Http/Middleware/*`, registered in `bootstrap/app.php`

Type	Class	Registered with	Runs on
Global	`AssignRequestId`	`$middleware->append()`	every request
Group	`ForceJsonResponse`	`$middleware->api(prepend: [...])`	every route in `routes/api.php`
Route	`EnsureUserHasRole`	`$middleware->alias(['role' => ...])`	only routes using `role:admin`
Route (built-in)	`auth:sanctum`, `throttle`, `can`	–	chosen routes

How middleware works: Laravel's **Pipeline** passes the request through each layer. Each layer can stop the request (`abort(403)`) or call `$next($request)`. Code **after** `$next()` runs on the way back, which is how `AssignRequestId` adds the header to the

response.

Say: "Middleware is for checks that apply to a whole group of routes. A policy is for checks about one specific record. In this project, `role:admin` protects the admin group, and the policy decides if a user can cancel *this* shipment."

Follow-ups:

- *What does `role:admin,manager` do?* — Values after the colon are passed as extra parameters (`string ...$roles`).
- *Why prepend ForceJsonResponse?* — So it runs before `auth:sanctum`. Then even a 401 comes back as JSON, not a redirect to a login page.

3.3 Validation

Files: `app/Http/Requests/*`, plus inline `$request->validate()` in `ShipmentController::index`

- **Form Request** classes keep validation out of the controller. If validation fails, Laravel automatically returns **422** with an `errors` object.
- `messages()` gives friendly custom messages (Pakistani mobile format).
- `Rule::enum(ShipmentStatus::class)` only accepts real status values.
- `$request->validated()` returns **only** the fields that have rules, so extra fields like `price` or `status` sent by a hacker are ignored.

Say: "I use Form Requests for create/update and inline validation for simple query filters. Price is never taken from the request — it's calculated on the server."

3.4 Authorization — policies

File: `app/Policies/ShipmentPolicy.php`, registered in `ShipmentServiceProvider::boot()`

The policy is used in **four different ways** in this project, which is a good thing to point out:

Where	Code	Ability
Form Request	`$this->user()->can('create', Shipment::class)`	`create`
Controller	`Gate::authorize('view', $shipment)`	`view`
Route middleware	`->can('cancel', 'shipment')`	`cancel`
API Resource	`$request->user()?->can('cancel', $this->resource)`	sent to Angular

`cancel()` returns `Response::deny('...')`, so the 403 message explains **why** it was refused.

Say: "Authentication asks *who are you*. Authorization asks *what are you allowed to do*. I return the permissions in the resource as a `can` block, so Angular shows or hides buttons using the same rules as the server. The server still checks everything — the front end is only for user experience."

3.5 Service container and service provider

File: `app/Providers/ShipmentServiceProvider.php` (added to `bootstrap/providers.php`)

This is where you show architecture knowledge. There are **four different container techniques** in this project:

What	Where	Why
Interface → class binding	`bind(CourierGateway::class, ...)`	Code depends on the interface. Config decides Fake or Http.
Singleton with a closure	`singleton(RateCalculator::class, ...)`	The constructor needs plain numbers from config. The container can't guess numbers, so we tell it how to build it.
Auto-wiring	`ShipmentService` (not registered at all)	Every constructor parameter is a class, so the container builds it on its own.
Method injection	`ShipmentStatusController::__invoke(..., ShipmentService $shipments)` and `BookShipmentWithCourier::handle(CourierGateway $courier, ...)`	The container also injects into methods it calls for you.

How the container builds `ShipmentController` (say this step by step):

1. The router needs `ShipmentController`. It asks the container: `make(ShipmentController::class)`.
2. No binding exists for it, so the container uses **PHP Reflection** to read the constructor: it needs `ShipmentService`.
3. It resolves `ShipmentService` the same way. Reflection shows that it needs `RateCalculator`.
4. `RateCalculator` **has a singleton binding**, so the container runs the closure once (reading config) and keeps that object.
5. It creates `new ShipmentService($rateCalculator)`, then `new ShipmentController($shipmentService)`.

In the queue worker, `handle(CourierGateway $courier)` asks for an **interface**. The container finds the binding and runs the closure, which returns `FakeCourierGateway` or `HttpCourierGateway`. Without that binding, Laravel would throw *"Target [CourierGateway] is not instantiable"*.

register() vs boot():

- `register()` only puts things **into** the container (`bind`, `singleton`).
- `boot()` runs after **all** providers are registered, so it can safely **use** other services. That's why the observer, the policy and the event listeners are set up in `boot()`.

Say: "To add a new courier like Leopards, I'd write one class that implements `CourierGateway` and add one line in the provider. No controller, job or service changes. That's the Open/Closed and Dependency Inversion principles from SOLID."

3.6 OOP in this project (useful for OOP questions)

Idea	Example in the project
Abstraction — hide *how* from the caller	`CourierGateway` interface. The job calls `book()` and never knows which company, URL or API key is used.
Encapsulation — protect the data from outside code	`status`, `tracking_number` and `price` are not fillable. They can only change through the observer, `ShipmentService` and the job. Status rules live inside the `ShipmentStatus` enum (`canMoveTo()`). Constructor properties are `private`.
Polymorphism — same method, different behaviour	`FakeCourierGateway::book()` vs `HttpCourierGateway::book()`.
Single Responsibility	Controller = HTTP only, Service = business rules, Job = slow work, Listener = one reaction each.

3.7 Observer

File: `app/Observers/ShipmentObserver.php`

- `creating` — **before** insert: sets the tracking number and the default status (values a user must never choose).
- `created` — **after** insert: fires the event for the first history row.
- `updated` — **after** update: fires the event only if `wasChanged('status')` is true. `getOriginal('status')` still gives the old value at this point.

Say: "I put this in an observer because it reacts to the *model*, not the controller. It doesn't matter if the status is changed by the API, the queue job, a seeder or tinker — the history is always written."

Follow-up: *What's the risk with observers?* — They're hidden, so someone debugging may not know they run. Heavy work in an observer slows every save. That's why the observer only fires an event and the slow work is queued. Also, `Model::query()->update()` (mass update) does **not** fire model events.

3.8 Events and listeners

Files: `app/Events/ShipmentStatusChanged.php`, `app/Listeners/*`, registered in `ShipmentServiceProvider::boot()`

Listener	Type	Why
`RecordStatusHistory`	**sync**	History must be saved together with the status change, inside the same transaction.
`SendStatusNotification`	**queued** (`ShouldQueueAfterCommit`, queue `notifications`)	Email is slow and can fail. It only goes to the queue once the transaction is saved.

Say: "An event says *something happened*. Each listener does one job. If tomorrow we need an SMS or a webhook to the client's system, I add one more listener and don't touch the existing code."

Why is event discovery turned off? In `bootstrap/app.php` I call `withEvents(discover: false)`, and I register the listeners by hand in the provider. That way the whole event map is visible in one place, and listeners can't be registered twice by accident.

3.9 Queue and job

File: `app/Jobs/BookShipmentWithCourier.php`

Setting	Value	Meaning
`$tries`	3	Try 3 times in total, then call `failed()`.
`backoff()`	10, 30, 60	Wait longer between each retry. Good for an API that's temporarily down.
`$timeout`	30	Kill the job if it runs over 30 seconds. `retry_after` in `config/queue.php` (90) must be **bigger** than this, or a slow job could run twice.
`WithoutOverlapping($id)`	–	Two workers can never book the same shipment at the same time.
Idempotent check	`status !== Pending -> return`	If the job runs twice, the second run does nothing.
`afterCommit()`	–	The job is only queued once the shipment row is saved.
`onQueue('courier')`	–	Separate queues, so a slow courier API can't delay emails.
`failed()`	–	Logs the error after the last try.

Say: "The courier API is slow and sometimes down, so the user should never wait for it. The request returns 201 straight away, and the job retries with backoff. I made it idempotent because in production any job can run more than once."

Try it live: set `COURIER_FAIL_RATE=0.5` and watch the worker retry.

3.10 Database design

Files: `database/migrations/2026_*`

- `price` and `weight_kg` are `DECIMAL`, never `FLOAT`, because float rounding breaks money.
- `tracking_number` has a **unique** index.
- Composite indexes match the real queries: `(user_id, status, created_at)` for the customer list and `(status, created_at)` for the admin list and dashboard. Equality columns come first, then the sort column.
- `shipment_status_histories` is insert-only, so it's a full audit trail.
- The dashboard uses **one** `GROUP BY` query instead of five `COUNT` queries.
- `show()` uses `load('histories.changedBy')` (eager loading) to avoid the N+1 problem.

3.11 API Resources

Files: `app/Http/Resources/*`

Resources decide the exact JSON shape. The database can change without breaking Angular. I group fields (`sender`, `receiver`, `status.value`/`status.label`), use ISO dates, and add the `can` permission block.

3.12 Tests

Files: `tests/Feature/ShipmentApiTest.php`, `tests/Unit/RateCalculatorTest.php`

They cover: guest gets 401, create works and the job is queued (`Queue::fake()`), validation returns 422, a customer can't see another customer's shipment (403), a customer can't use admin routes (403), an invalid status move returns 422, and the job books the shipment. The unit test checks the price calculation with no database at all — that's easy because `RateCalculator` takes plain values in its constructor.

---

4. Live demo script (curl)

```
# Login as customer and keep the token
TOKEN=$(curl -s -X POST http://localhost:8000/api/auth/login \
  -H "Content-Type: application/json" \
  -d '{"email":"customer@example.com","password":"password"}' | php -r 'echo
json_decode(stream_get_contents(STDIN))->token;')

# Live price
curl -s -X POST http://localhost:8000/api/shipments/quote -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{"weight_kg":2,"sender_city":"Lahore","receiver_city":"Karachi"}'

# Create a shipment  -> watch the queue worker terminal book it
curl -s -X POST http://localhost:8000/api/shipments -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  -d
'{"sender_name":"Sajid","sender_city":"Lahore","receiver_name":"Ahmed","receiver_phone":"03001234567","receiver_addr
 1","receiver_city":"Karachi","weight_kg":2}'

# Validation error (422)
curl -s -X POST http://localhost:8000/api/shipments -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{"receiver_phone":"123","weight_kg":99}'

# Customer tries an admin route (403)
curl -s -X PATCH http://localhost:8000/api/admin/shipments/1/status -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" -d '{"status":"in_transit"}'

# Shipment detail with history
curl -s http://localhost:8000/api/shipments/1 -H "Authorization: Bearer $TOKEN"
```

---

5. "What would you change for production?"

Interviewers often ask this. Good answers:

1. **Redis + Horizon** instead of the database queue, with separate supervisors for `courier` and `notifications`, and alerts on queue wait time.
2. **Webhooks from the courier** for in-transit/delivered updates instead of an admin changing them by hand. Verify the webhook signature and handle each one idempotently.
3. **Rate limiting the courier API** with the `RateLimited` job middleware.
4. **Monitoring:** Sentry for errors, and the request ID in every log line (already done).
5. **API versioning** (`/api/v1`) before the mobile app goes live.
6. **Cursor pagination** if the shipments table grows to millions of rows.
7. **A reconciliation command** on the scheduler that finds shipments stuck in `pending` for too long and re-queues them.

---

6. Be honest about it

If they ask: "This is a small demo project I built to show how I structure a Laravel feature. In my real jobs I've done the same patterns on larger systems — for example [your real project]." Then connect it to a real story about queues, a slow query or a payment integration.

Setup & API Reference (README)

A small courier/logistics API. A customer books a parcel, the system calculates the price, books it with a courier company in the background, and keeps a full status history. An admin moves the parcel through its statuses.

The project is intentionally small: **one feature, built properly**. It shows authentication, authorization (policies), all three kinds of middleware, validation, a custom service provider with dependency injection, queues/jobs, an observer, events and listeners, and a clean JSON API ready for an Angular front end.

> For a file-by-file explanation and interview talking points, read **docs/INTERVIEW_GUIDE.md**.

---

1. Requirements

- PHP 8.3+
- Composer 2
- MySQL 8 (or MariaDB 10.6+)
- Node is **not** needed (this is an API only)

2. Setup

```
# 1. Install PHP packages
composer install

# 2. Create your .env and app key
cp .env.example .env
php artisan key:generate

# 3. Create the database in MySQL
mysql -u root -p -e "CREATE DATABASE shipment_api CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;"
#    then check DB_USERNAME / DB_PASSWORD in .env

# 4. Create tables and demo data
php artisan migrate --seed
```

Demo users (password for both: `password`):

Role	Email
Admin	admin@example.com
Customer	customer@example.com

3. Run it

You need **two terminals**: one for the web server, one for the queue worker.

```
# Terminal 1 — the API on http://localhost:8000
php artisan serve

# Terminal 2 — the queue worker (handles courier booking + emails)
php artisan queue:work --queue=courier,notifications,default
```

Emails use `MAIL_MAILER=log`, so you can read them in `storage/logs/laravel.log`.

To see **retries** in action, set `COURIER_FAIL_RATE=0.5` in `.env`, restart the worker and create a few shipments.

4. Run the tests

Tests use SQLite in memory (see `phpunit.xml`), so they don't touch your MySQL data.

```
php artisan test
```

---

5. API reference

Base URL: `http://localhost:8000/api` Send `Authorization: Bearer <token>` on every route except register and login. All responses are JSON. Every response has an `X-Request-Id` header.

Auth

Method	URL	Body	Notes
POST	`/auth/register`	`name, email, password, password_confirmation`	Returns `user` + `token`. Max 10/min.
POST	`/auth/login`	`email, password, device_name?`	Returns `user` + `token`. Max 5/min.
GET	`/auth/me`	–	Current user.
POST	`/auth/logout`	–	Deletes the current token. Returns 204.

Login response:

```
{
  "user": { "id": 2, "name": "Ali Customer", "email": "customer@example.com", "role": "customer" },
  "token": "3|Qm9vZ3Vz..."
}
```

Shipments

Method	URL	Who	Notes
GET	`/dashboard`	logged in	Counts per status. Customers see only their own.
POST	`/shipments/quote`	logged in	Price only, nothing saved. For the live price on the form.
GET	`/shipments`	logged in	Paginated. Filters: `status`, `search`, `per_page`.
POST	`/shipments`	logged in	Creates a shipment and queues the courier booking. 201.
GET	`/shipments/{id}`	owner or admin	Includes `history`.
POST	`/shipments/{id}/cancel`	owner or admin	Only `pending` or `booked`.
PATCH	`/admin/shipments/{id}/status`	**admin only**	Body: `{ "status": "in_transit" }`

Create shipment — request:

```
{
  "sender_name": "Sajid",
  "sender_city": "Lahore",
  "receiver_name": "Ahmed",
  "receiver_phone": "03001234567",
  "receiver_address": "House 1, Street 2, DHA",
  "receiver_city": "Karachi",
  "weight_kg": 2
}
```

Shipment — response:

```
{
  "data": {
    "id": 12,
    "tracking_number": "SHP261019K4P9ZQ",
    "sender":   { "name": "Sajid", "city": "Lahore" },
    "receiver": { "name": "Ahmed", "phone": "03001234567", "address": "House 1, Street 2, DHA", "city": "Karachi"
},
    "weight_kg": 2.0,
    "price": 750.0,
    "status": { "value": "pending", "label": "Pending" },
    "courier_reference": null,
    "booked_at": null,
    "created_at": "2026-10-19T06:00:00+00:00",
    "can": { "cancel": true, "update_status": false },
    "history": [
      { "from": null, "to": "pending", "to_label": "Pending", "changed_by": "Ali Customer", "at": "2026-10-
19T06:00:00+00:00" }
    ]
  }
}
```

The `can` block tells the Angular app which buttons to show, so the front end never repeats the permission rules.

Quote — response:

```
{ "data": { "base_fee": 200, "weight_charge": 300, "same_city": false, "total": 750 } }
```

Dashboard — response:

```
{ "data": { "total": 11, "by_status": { "pending": 3, "booked": 2, "in_transit": 2, "delivered": 3, "cancelled": 1
} } }
```

List — response: `data` (array of shipments), plus Laravel's `links` and `meta` (`current_page`, `last_page`, `per_page`, `total`).

Status rules

```
pending ---> booked ---> in_transit ---> delivered
   |            |
   +------------+---> cancelled
```

An invalid move (for example `pending -> delivered`) returns **422** with a clear message.

Error format

Code	When	Example body
401	Missing or wrong token	`{ "message": "Unauthenticated." }`
403	Not allowed (policy or role)	`{ "message": "Only pending or booked shipments can be cancelled." }`
404	Shipment not found	`{ "message": "..." }`
422	Validation failed	`{ "message": "...", "errors": { "receiver_phone": ["Enter a valid mobile number like 03001234567."] } }`
429	Too many login attempts	`{ "message": "Too Many Attempts." }`

---

6. Using it from Angular

- CORS allows `FRONTEND_URL` (default `http://localhost:4200`). Change it in `.env`.

- Save the `token` after login, and add it to every request with an **HTTP interceptor**: `Authorization: Bearer <token>`.
- On a **401**, the interceptor should clear the token and send the user to `/login`.
- On a **422**, read `error.error.errors` and show each message under the matching form field.
- Suggested screens: Login, Dashboard, Shipments list with filters, New shipment form with live price, Shipment detail with history and a Cancel button (`can.cancel`), and an admin status dropdown (`can.update_status`).

Example TypeScript types:

```
export type ShipmentStatus = 'pending' | 'booked' | 'in_transit' | 'delivered' | 'cancelled';

export interface Shipment {
  id: number;
  tracking_number: string;
  sender: { name: string; city: string };
  receiver: { name: string; phone: string; address: string; city: string };
  weight_kg: number;
  price: number;
  status: { value: ShipmentStatus; label: string };
  courier_reference: string | null;
  booked_at: string | null;
  created_at: string;
  can: { cancel: boolean; update_status: boolean };
  history?: { from: ShipmentStatus | null; to: ShipmentStatus; to_label: string; changed_by: string; at: string }
[];
}

export interface Paginated<T> {
  data: T[];
  meta: { current_page: number; last_page: number; per_page: number; total: number };
}
```

---

7. Project map

```
app/
  Contracts/CourierGateway.php              interface (abstraction)
  Enums/ShipmentStatus.php, UserRole.php    statuses + transition rules
  Events/ShipmentStatusChanged.php
  Exceptions/CourierUnavailableException.php
  Http/
    Controllers/Api/                        AuthController, ShipmentController, DashboardController
    Controllers/Api/Admin/                  ShipmentStatusController (invokable)
    Middleware/                             AssignRequestId (global), ForceJsonResponse (group), EnsureUserHasRole
(route)
    Requests/                               Form Request validation
    Resources/                              JSON output shape
  Jobs/BookShipmentWithCourier.php          queued job with retries
  Listeners/                                RecordStatusHistory (sync), SendStatusNotification (queued)
  Models/                                   User, Shipment, ShipmentStatusHistory
  Notifications/ShipmentStatusUpdated.php
  Observers/ShipmentObserver.php
  Policies/ShipmentPolicy.php
  Providers/ShipmentServiceProvider.php     container bindings, observer, policy, events
  Services/                                 ShipmentService, RateCalculator, Courier/{Fake,Http}CourierGateway
bootstrap/app.php                           middleware setup, api routes, event discovery off
config/shipping.php                         prices + courier driver
routes/api.php
tests/                                      Feature + Unit tests
```

8. Switching to Redis + Horizon (production)

The project uses the `database` queue so it runs with no extra setup. For production:

```
composer require laravel/horizon
php artisan horizon:install
```

Then set `QUEUE_CONNECTION=redis`, add supervisors for the `courier` and `notifications` queues in `config/horizon.php`, run `php artisan horizon` instead of `queue:work`, and run `php artisan horizon:terminate` after every deploy.